Homework 1: BIST and BISR
Student reference

Cedar-8f2c

Student reference. This is an anonymous, editable reconstruction of the supplied student report. It preserves the student's approach and reported results. Editorial verification notes and assignment clarifications are labeled separately. It is not a newly corrected solution or a claim of full fault coverage. The accompanying anonymized PDF preserves the original evidence.

1 Assignment and supplementary requirements

The assignment asks for a 16-bit ripple carry adder with built-in self-test (BIST), followed by a repairable version with one spare full adder. The BIST should detect single cell faults. The repair design should locate a faulty full adder and restore normal arithmetic. The supplementary instructions require test-mode patterns to originate inside the BIST. An external testbench may supply operands only in normal mode. Fault injection must support every one of the 16 full-adder positions, with one faulty cell at a time. The report should show normal arithmetic, a fault changing the result, detection in test mode, and arithmetic after repair. A representative waveform is permitted, but the intended coverage still applies to every supported fault.

2 Part 1: BIST architecture

The student places a pattern source and response comparator in front of a 16-bit ripple carry adder. The signals `Test`, `En`, `Clk`, and `Rst` select normal operation or the embedded pattern sequence. The 17-bit response includes all sum bits and the final carry. The signal `P_F` is zero for pass and becomes one after a mismatch.

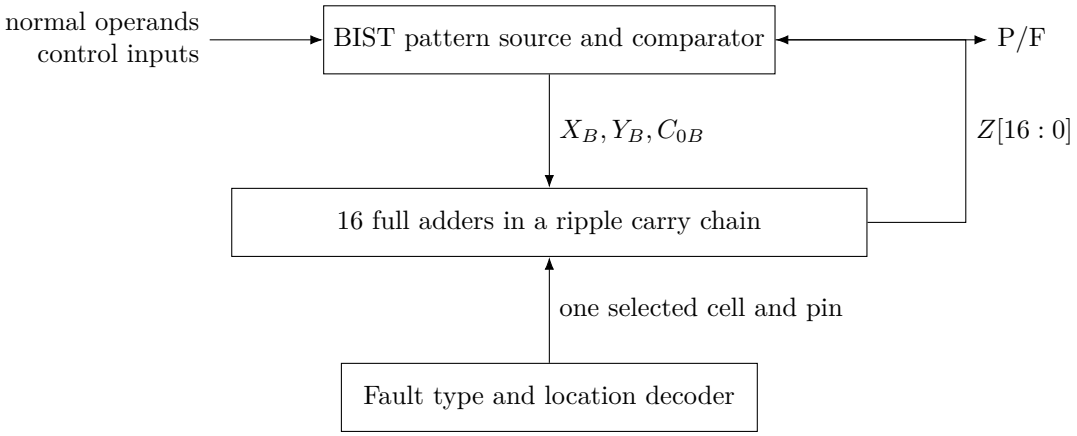


Figure 1: Editable reconstruction of the source presentation's BIST architecture.

2.1 Pattern selection and comparison

Normal mode forwards the external operands and carry input. Test mode with `En=0` holds the embedded operands. With `En=1`, the design alternates all-zero and all-one operands, including the

carry input, and compares the response with the expected value. The implementation pipelines the comparison. Reset clears the failure flag and pattern counters.

Mode	Operand source	Action
Test=0	External X, Y, C_0	Normal arithmetic
Test=1, En=0	Held embedded values	Hold sequence
Test=1, En=1	Alternating embedded patterns	Compare response
Mismatch	–	Set sticky P/F failure flag

2.2 Fault model and pattern argument

The student uses ten pin stuck-at faults per full adder:

$$\{X/0, X/1, Y/0, Y/1, C_{\text{in}}/0, C_{\text{in}}/1, S/0, S/1, C_{\text{out}}/0, C_{\text{out}}/1\}.$$

The source report gives this complete sum-output table. A fault column forces the named pin to the stated value.

X	Y	C_{in}	S	$X/0$	$X/1$	$Y/0$	$Y/1$	$C_{\text{in}}/0$	$C_{\text{in}}/1$	$S/0$	$S/1$
0	0	0	0	0	1	0	1	0	1	0	1
0	0	1	1	1	0	1	0	0	1	0	1
0	1	0	1	1	0	0	1	1	0	0	1
0	1	1	0	0	1	1	0	1	0	0	1
1	0	0	1	0	1	1	0	1	0	0	1
1	0	1	0	1	0	0	1	1	0	0	1
1	1	0	0	1	0	1	0	0	1	0	1
1	1	1	1	0	1	0	1	0	1	0	1

The report argues that $(0, 0, 0)$ detects the five stuck-at-one faults and $(1, 1, 1)$ detects the five stuck-at-zero faults when sum and carry are observed. For the whole adder these patterns become

$$(X, Y, C_0) = (0, 0, 0), \qquad (65535, 65535, 1),$$

with expected 17-bit responses 0 and 131071.

Editorial verification note. This argument concerns the ten stated pin stuck-at faults. General single-cell functional faults can alter an arbitrary truth-table entry. Two input combinations do not establish coverage of that broader model. The report's coverage claim therefore requires this qualification.

2.3 Fault injection

The four-bit `Fault_loc` chooses one of 16 full adders. The upper five bits of `Fault_type` select a pin, and bit zero gives its forced value.

Fault_type[5:1]	Selected pin
00000	No fault
10000	X
01000	Y
00100	C_{in}
00010	S
00001	C_{out}

Only one pin selection is intended at a time. Bit zero equal to zero means stuck-at-zero, while bit zero equal to one means stuck-at-one.

2.4 Reported waveform example

The report first demonstrates normal arithmetic. It injects a Y stuck-at-zero fault at full adder 4. For $X = 1300$, $Y = 56$, and $C_0 = 1$, the correct result is 1357 and the faulty result is 1341. Entering test mode and enabling embedded patterns makes `P_F` become one.

Stage	Test	Fault	Reported observation
Normal arithmetic	0	None	Correct sums
Injection	0	FA4, $Y/0$	$1357 \longrightarrow 1341$
Embedded test	1	FA4, $Y/0$	Mismatch, P/F=1

This event table reconstructs the source waveform's explanation. It is not a new simulation trace.

3 Part 2: BISR

The design adds one spare full adder and inserts its sum and carry into the chain after locating a fault. It compares the observed response with the expected response:

$$D = Z_{\text{compare}} \oplus Z_{\text{correct}}.$$

A priority decoder examines D from the least significant bit toward the most significant bit. The first nonzero bit selects the full adder to replace. A mismatch only in the final carry selects full adder 15. Once failure is asserted, location-dependent logic sends the selected stage's original operands

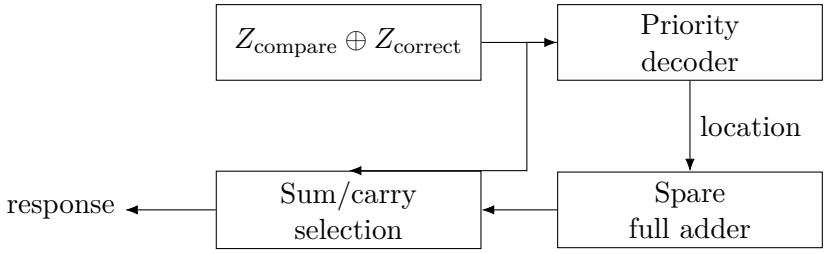


Figure 2: Editable reconstruction of the fault-location and spare-selection method.

and incoming carry to the spare. It replaces that stage's sum and drives the next stage from the spare carry. The source report shows excerpts of the `casez` priority decoder and the replacement `case` statement. The complete implementation is retained in the accompanying Verilog.

The final reported waveform repeats normal operation and injection, identifies the faulty location, and demonstrates correct arithmetic with the spare.

Editorial verification note. The first erroneous sum bit need not identify the physical source of every carry-output fault. Each stage and fault type requires directed testing. This reconstruction preserves the student's claim, while the repository's verification notes report observed limitations separately.

4 Source selection

The submitted `hw1_1.v`, `hw1_1_tb.v`, `hw1_2.v`, and `hw1_2_tb.v` are the primary set. A second top-level source copy is identical. Distinct files from two project source directories remain in separate legacy directories. Reused module names require separate compilation. Project files, generated simulation files, and caches are unnecessary.

5 Repository verification

Icarus Verilog 13.0 executed the retained RTL and the supplied testbenches. A separate testbench checks 100 normal operand combinations and all $16 \times 5 \times 2 = 160$ supported pin stuck-at injections. The BIST has no normal-arithmetic failures, keeps P/F zero on a good circuit, and detects all 160 injected faults under this sequence. The BISR also detects all 160 injections, but its decoded location differs from the injected cell in 34 cases. After each test, a normal-mode calculation of 1300+56+1 is checked. It returns an incorrect result for 37 of the 160 injections. These are observed limitations of the preserved reference, not edits to the student's submission. This directed test does not establish correctness for every post-repair operand or a general cell-functional fault model. The reproducible driver is `verification/run_checks.py` relative to the homework collection.